

1.

(1)

在 NexOS 中新增一个用户程序所需的三个必要步骤：

1. 在 `user/` 目录下编写该程序对应的 `.c` 源文件；
2. 在根目录 `Makefile` 的 `UPROGS` 中添加该程序名称；
3. 在根目录 `Makefile` 的 `fsimg` 规则中，将该程序打包进 `fs.img`。

(2)

如果未向 `UPROGS` 中加入 `sid`，则 `sid.c` 不会被编译；如果没有更新 `fsimg` 规则，则编译得到的 `sid.elf` 不会被打包进 `fs.img`，从而无法在 NexOS Shell 中运行。

2.

内建命令由 Shell 自行处理，而外部命令由 Shell 通过创建子进程 `fork + exec` 后执行，以独立可执行文件的形式存在。`cd` 的作用对象是 Shell 本身的路径，因此必须由 Shell 自己执行；而 `echo/ls` 等命令的执行效果与 Shell 本身无关，因此可以作为外部命令存在。

3.

输入重定向：把文件接到标准输入（`stdin`，文件描述符 `fd=0`）；输出重定向：把文件接到标准输出（`stdout`，文件描述符 `fd=1`）。

4.

(1)

添加系统调用的过程（以注册新系统调用 `hello_id(int tag)` 为例）

1. 在 `kernel/include/syscall.h` 中注册新系统调用号 25，注意**不应与已有的系统调用号冲突**；
2. 在 `kernel/core/syscall.c` 中实现对应的内核函数；
 - 对于前三个传入参数，分别直接访问 `a0/a1/a2` 寄存器即可获得：`p->trapframe->a0/a1/a2`
 - 寄存器是 64 位，缺省对应 `uint64` 格式

```
◦ static uint64 sys_hello_id(void) {
    struct proc *p = myproc();
    if (p == 0) {
        return (uint64)-1;
    }

    // 从 trapframe 的 a0 寄存器中取出第一个参数
    int tag = (int)p->trapframe->a0;

    printf("[sys_hello_id] pid=%d, tag=%d\n", p->pid, tag);

    // 将 tag 作为返回值返回给用户态
```

```
        return (uint64)tag;
    }
```

3. 在 `kernel/core/syscall.c` 的系统调用分发表中注册新系统调用，把添加的系统调用号映射到对应的实现函数上

- `[SYS_hello_id] = sys_hello_id`

4. 在 `user/syscall.c` 中添加用户态封装

```
◦ int hello_id(int tag) {
    return (int)__syscall(SYS_hello_id, tag, 0, 0);
}
```

5. 在 `user/user.h` 中声明接口

- `int hello_id(int tag);`

(2)

在调用系统调用的过程中，用户态参数通过 `a0/a1/a2` 寄存器传递，系统调用号通过 `a7` 寄存器传递，返回值通过 `a0` 寄存器反馈给调用方。